COMPUTACIÓN Y ESTRUCTURAS DISCRETAS III · INTEGRATIVE TASK 1

ResumeLens

Formal language-based résumé screening

Regular expressions · finite-state transducers · finite automata · context-free grammars

Juan Andrés Cano Torres
Samuel Cardoso Martinez
Juan Felipe Martinez

Universidad Icesi · Group 3
Professor Andrés · 2026-2

THE PROBLEM

One qualification, many spellings

JS Javascript JavaScript	sklearn scikit learn Scikit-learn	Postgres PostgreSQL postgres
⬇	⬇	⬇
JAVASCRIPT	**SCIKIT_LEARN**	**POSTGRESQL**

Goal: decide whether the qualifications **explicitly written** in a résumé satisfy a formally defined pattern for a professional profile.

Not a ranking and not a hiring decision — every verdict must be explainable.

METHODOLOGY

Four stages, four formal models

STAGE 1

Extract

Regular expressions

`Python re`

Out: raw strings such as “React.js”

STAGE 2

Normalize

Finite-state transducer

`pyformlang`

Out: canonical tokens in profile order

STAGE 3

Recognize

Finite automata (ε-NFA → DFA)

`pyformlang`

Out: ACCEPTED or REJECTED per profile

STAGE 4

Structure

Context-free grammar

`textX`

Out: validated profile and HTML page

Profiles are data — ordered categories of tokens. One implementation serves all four:

Full Stack Developer | Machine Learning Engineer | DevOps Engineer · team-defined | Data Engineer · team-defined

STAGE 1 · REGULAR EXPRESSIONS

Extraction only finds candidate strings

Mary Jane Watson
2 years of experience developing **predictive models** and data-processing pipelines.
Technical Skills:
Python, **Pandas**, **NumPy**, **Scikit-learn**, **TensorFlow**, **SQL**, **Git**.

name · **experience** · **skill** (six regex families)

```
(?<![\w.])
(?: pandas | numpy
  | scikit[ -]?learn | sklearn
  | tensor[ -]?flow | py[ -]?torch
  | keras )
(?!\w)
```

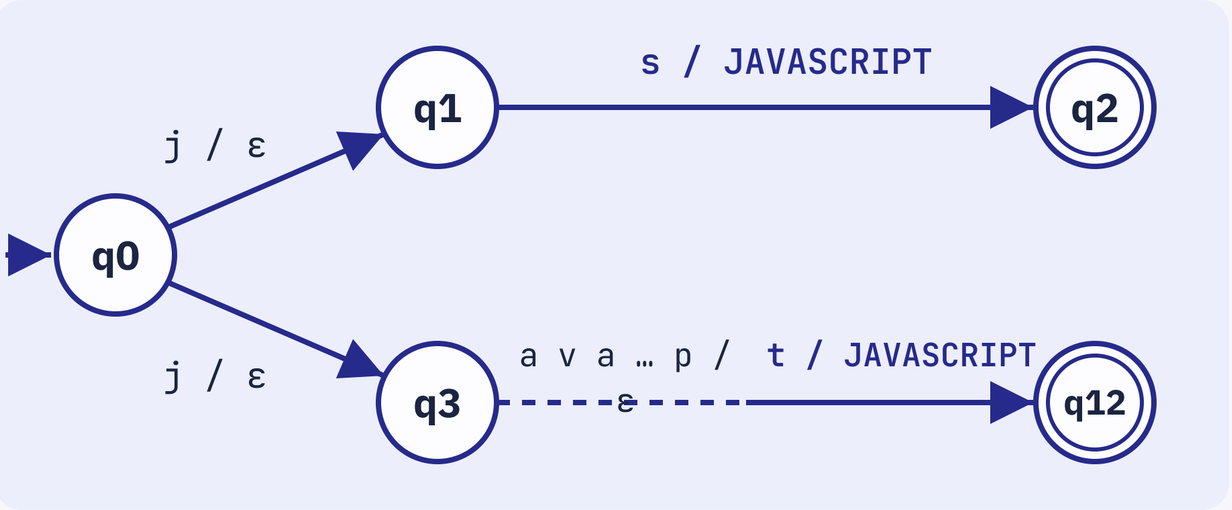
The library family, case-insensitive. Others: languages, frameworks, databases, tools, practices — plus name, e-mail, phone, links, education and experience.

The **boundaries** stop js inside NodeJS and SQL inside PostgreSQL from being reported.

The **original spelling is kept**: deciding equivalence is stage 2’s job.

STAGE 2 · FINITE-STATE TRANSDUCER

Every spelling becomes one token



a / b = read a, write b. Double circle = accepting state. Both paths write **JAVASCRIPT**.

Design decision: Σ = characters after case folding, so one table covers JS, Js and js — **106 variants → 48 tokens**.

M	M = (Q, Σ, Γ, δ, ω, q0, F)
Q	q0 plus one state per character of each variant (893)
Σ	characters of the case-folded variants (30)
Γ	canonical skill tokens (48)
δ	follows a variant's path, one character at a time
ω	ε on every step except the last, which writes the token
q0	shared initial state
F	last state of every variant path

STAGE 2 · CANONICAL ORDER

Sorting makes the verdict order-independent

AS WRITTEN	`Git, NodeJS, JS, Postgres, React.js`
NORMALIZED	`GIT, NODE_JS, JAVASCRIPT, POSTGRESQL, REACT`
SORTED FOR FULL STACK	`JAVASCRIPT, REACT, NODE_JS, POSTGRESQL, GIT`

Full Stack category order

Frontend language → Frontend framework → Backend → Database → API *(optional)* → Version control

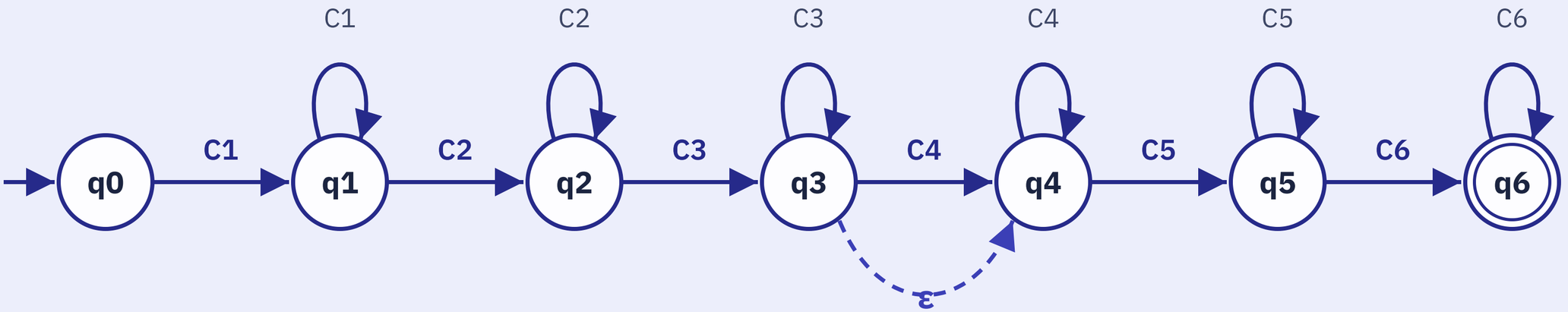
Projection and deduplication

Tokens outside a profile's alphabet are dropped; JS and JavaScript in one résumé give a single JAVASCRIPT.

STAGE 3 · FINITE AUTOMATA

Each profile is a regular language

L(P) = X1 X2 … Xn, where **Xi = Ci⁺** if category i is required and **Xi = Ci*** if it is optional



C1 PYTHON

C2 PANDAS, NUMPY

C3 SCIKIT_LEARN, TENSORFLOW, PYTORCH, KERAS

C4 ML_MODEL_DEVELOPMENT *(optional)*

C5 SQL, POSTGRESQL, MYSQL

C6 GIT

STAGE 3 · TYPE OF AUTOMATON

Why an ε-NFA — and why that is enough

1. Optional categories are skipped with **ε-transitions**, so the machine as built is not a DFA.
2. Categories are **disjoint**: each token has at most one move, so ε is the only non-determinism.
3. Subset construction and minimization give an **equivalent minimal DFA** with the same n + 1 states — tested for all four profiles.

Profile	|Q|	|Σ|	ε
Full Stack Developer	7	19	1
Machine Learning Engineer	7	12	1
DevOps Engineer	9	14	2
Data Engineer	7	18	1

ASSIGNMENT EXAMPLE

```
PYTHON PANDAS TENSORFLOW POSTGRESQL GIT
q0 → q1 → q2 → q3 -ε→ q4 → q5 → q6
```

Accepted: q6 is the accepting state

STAGE 4 · CONTEXT-FREE GRAMMAR · TEXTX

A small language for the candidate profile

```
Candidate = "candidate" STRING "{"
  Contact {Education} {Experience}
  Skills Classification "}" ;
Contact = "contact" "{" {ContactItem} "}" ;
ContactItem = ("email"|"phone"|"link")
  ":" STRING ;
Experience = "experience" "{" "years" ":" INT
  ["description" ":" STRING] "}" ;
Skills = "skills" ":" TOKEN {"," TOKEN} ;
Classification = "classification" "{"
  ProfileResult {ProfileResult} "}" ;
ProfileResult = PROFILE_KEY ":" VERDICT ;
TOKEN = [A-Z] {[A-Z0-9_]} ;
```

```
candidate "Wednesday Addams" {
  contact { }
  experience { years: 3 }
  skills: JAVASCRIPT, REACT,
    NODE_JS, POSTGRESQL, GIT
  classification {
    FULL_STACK_DEVELOPER: ACCEPTED
    DATA_ENGINEER: REJECTED …
  }
}
```

```
skills: javascript
→ line 8, column 11: Expected TOKEN
```

Keywords are lowercase and tokens uppercase, so they never clash. A valid profile is rendered as an HTML page.

RESULTS

Verdicts for six sample résumés

Résumé	Full Stack	ML Engineer	DevOps	Data Engineer
Wednesday Addams	Accepted	—	—	—
Mary Jane Watson	—	Accepted	—	—
Alex Morgan	—	—	Accepted	—
Taylor Reed	—	—	—	Accepted
Sam Rivera · near miss	—	—	—	—
Jordan Lee · two profiles	—	—	Accepted	Accepted

Accepted = the profile’s automaton ends in its accepting state; — = rejected. The first four are the reference samples, one per profile; the shaded rows are edge cases added for evaluation.

WHAT THE RESULTS SHOW

A rejection always means a missing required category

Order and foreign tokens are removed before recognition, so the only way to fail is a required category C_i^+ with no token.

Near miss · Sam Rivera

`JAVASCRIPT TYPESCRIPT REACT VUE GIT`

Full Stack: q0 → q1 → q1 → q2 → q2, then GIT has no move from q2. Missing: Backend and Database.

Two profiles · Jordan Lee

`LINUX BASH DOCKER JENKINS AWS APACHE_SPARK …`

Covers every required category of DevOps Engineer and of Data Engineer. Profiles are evaluated independently.

VALIDATION

164 automated tests cover every stage

37	53	31	20	23
Extraction regular expressions	Normalization transducer, sorting	Automata and their diagrams	Grammar validation, HTML	End to end pipeline, CLI, report

Each **ε-NFA is equivalent to its minimal DFA**, for all four profiles.

Invalid profiles are rejected: lowercase tokens, missing braces, unknown profiles, empty text.

The **generated documentation** — automata definitions and results — must match the code.

LIMITATIONS AND FUTURE WORK

What the model does not capture

Limitations

- Unknown spellings, such as Rust, are ignored until added to the variant table.
- Only explicit skills count: no seniority, context or proficiency.
- Variants share no prefixes, so the transducer has 893 states.
- Skill patterns and profiles are maintained by hand.

Future work

- A prefix-sharing, deterministic transducer with an end marker.
- A separator-removing transducer composed before normalization.
- Skill vocabularies from ESCO or O*NET.
- Comparison with a learned extractor on annotated résumés.

CONCLUSIONS

The weakest model for each job

Regex finds, the transducer rewrites, automata recognize, the grammar structures.

Every verdict is explainable

An accepted path, or the required category that was missing.

Profiles are data

One construction builds the automata for all four profiles.

Thank you — questions?

github.com/Jacanotorres/ResumeLens-CyED3